

Guia Prático Avançado: Modernização e Teste de Estresse

Mini Radar ENEM — Simulação de PaaS, FaaS e Teste de Carga com Locust

Disciplina: Computação em Nuvem

Integrantes: João Pedro Silva, João Pedro Benvenuti, Gabriel Xavier, Nyrx

1. Introdução

Esta atividade dá continuidade ao projeto Mini Radar ENEM, evoluindo a aplicação de um monólito simples para uma arquitetura desacoplada em microsserviços. O objetivo é simular, em ambiente local, dois modelos de serviço em nuvem trabalhando em conjunto:

- **PaaS (Platform as a Service):** representado pela aplicação principal em Flask (`app.py`), responsável por exibir as informações ao aluno.
- **FaaS / Serverless (Function as a Service):** representado pelo microsserviço em FastAPI (`calculadora_service/main.py`), responsável exclusivamente por calcular a nota de corte.

Além disso, foi realizado um teste de carga com a ferramenta **Locust**, para avaliar o comportamento da arquitetura sob estresse e validar o mecanismo de **resiliência ativa (graceful degradation)** implementado no site principal.

2. Fase 1 — Baseline da Calculadora (FaaS com 1 worker)

Antes de simular a escala da nuvem, o microsserviço de cálculo foi executado com apenas **1 worker**, representando um servidor comum, sem paralelismo:

```
uvicorn calculadora_service.main:app --port 8000 --workers 1
```

Esse cenário serve como linha de base (baseline) para comparação: com um único processo atendendo às requisições, o serviço tem capacidade limitada para responder a múltiplos acessos simultâneos.

```
PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS
joao@JP-G5-5590:~/Documentos/Mini-Radar-ENEM-com-Python-e-Docker-Aula-3-Modelos-de-servico-IaaS-PaaS-SaaS$ source /home/joao/Documentos/Mini-Radar-ENEM-com-P
ython-e-Docker-Aula-3-Modelos-de-servico-IaaS-PaaS-SaaS/.venv/bin/activate
(.venv) joao@JP-G5-5590:~/Documentos/Mini-Radar-ENEM-com-Python-e-Docker-Aula-3-Modelos-de-servico-IaaS-PaaS-SaaS$ uvicorn calculadora_service.main:app --port
8000 --workers 1
INFO: Started server process [21840]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
```

Figura 1 — Inicialização da API de cálculo (FastAPI) com 1 worker, simulando um servidor tradicional sem escalabilidade.

3. Fase 2 — Escalando a Calculadora (FaaS com 4 workers)

Em seguida, o mesmo serviço foi reiniciado com **4 workers**, simulando o comportamento de um ambiente serverless na nuvem, no qual múltiplas instâncias da função são provisionadas automaticamente para atender à demanda:

```
uvicorn calculadora_service.main:app --port 8000 --workers 4
```

Como pode ser observado no terminal, o Uvicorn inicia um processo pai e **quatro processos filhos** (Started server process), cada um pronto para atender requisições de forma paralela — o equivalente local ao provisionamento automático de instâncias que a AWS ou o Azure realizariam em um ambiente real de FaaS.

```
PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS
(.venv) joao@JP-G5-5590:~/Documentos/Mini-Radar-ENEM-com-Python-e-Docker-Aula-3-Modelos-de-servico-IaaS-PaaS-SaaS$ uvicorn calculadora_service.main:app --port
8000 --workers 4
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started parent process [22145]
INFO: Started server process [22147]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Started server process [22148]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Started server process [22150]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Started server process [22149]
INFO: Waiting for application startup.
INFO: Application startup complete.
```

Figura 2 — API de cálculo reiniciada com 4 workers, simulando o provisionamento automático de instâncias em um ambiente Serverless/FaaS.

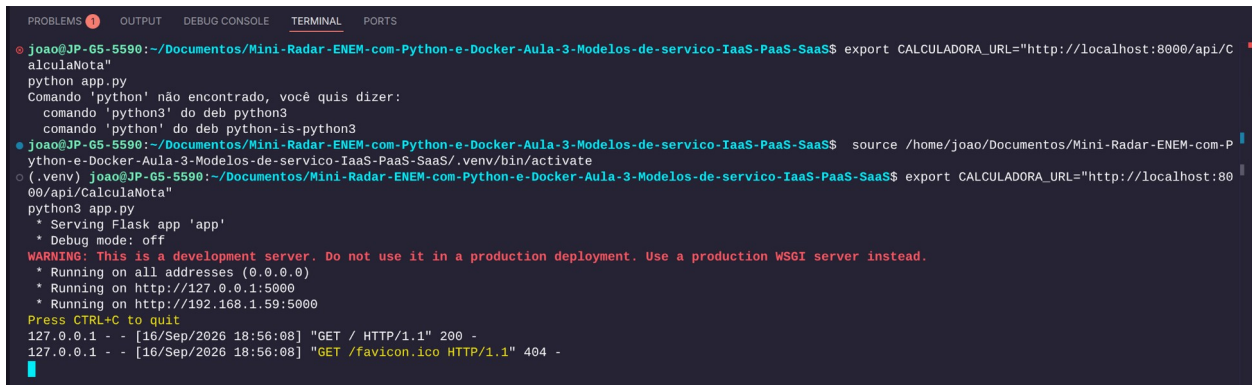
4. Fase 3 — Site Principal (PaaS) Consumindo o Microserviço

Com a calculadora no ar, a aplicação principal em Flask foi iniciada, configurada por variável de ambiente para localizar o microserviço — sem necessidade de alterar nenhuma linha de código:

```
export CALCULADORA_URL="http://localhost:8000/api/CalculaNota"
python3 app.py
```

O terminal confirma que o Flask subiu com sucesso (Serving Flask app) e respondeu a uma

requisição GET / com status 200.



```
joao@JP-G5-5590:~/Documentos/Mini-Radar-ENEM-com-Python-e-Docker-Aula-3-Modelos-de-servico-IaaS-PaaS-SaaS$ export CALCULADORA_URL="http://localhost:8000/api/C"
alculaNota"
python app.py
Comando 'python' não encontrado, você quis dizer:
comando 'python3' do deb python3
comando 'python' do deb python-is-python3
joao@JP-G5-5590:~/Documentos/Mini-Radar-ENEM-com-Python-e-Docker-Aula-3-Modelos-de-servico-IaaS-PaaS-SaaS$ source /home/joao/Documentos/Mini-Radar-ENEM-com-P
ython-e-Docker-Aula-3-Modelos-de-servico-IaaS-PaaS-SaaS/.venv/bin/activate
(.venv) joao@JP-G5-5590:~/Documentos/Mini-Radar-ENEM-com-Python-e-Docker-Aula-3-Modelos-de-servico-IaaS-PaaS-SaaS$ export CALCULADORA_URL="http://localhost:80
00/api/CalculaNota"
python3 app.py
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://192.168.1.59:5000
Press CTRL+C to quit
127.0.0.1 - - [16/Sep/2026 18:56:08] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [16/Sep/2026 18:56:08] "GET /favicon.ico HTTP/1.1" 404 -
```

Figura 3 — Aplicação principal (Flask/PaaS) em execução, configurada via variável de ambiente `CALCULADORA_URL` para se comunicar com o microserviço.

Para validar a integração entre os dois serviços, foi feita uma chamada ao endpoint `/exibir_nota`, que repassa as notas para a calculadora e devolve o resultado calculado:

```
curl -X POST http://localhost:5000/exibir_nota \
-H "Content-Type: application/json" \
-d '{"notas": [720.5, 680.0, 810.2, 640.8, 780.0]}'
```

A resposta obtida foi `{"mensagem": "Sua nota de corte é: 726.3"}`, confirmando a comunicação correta entre o site principal e o microserviço de cálculo.



```
joao@JP-G5-5590:~/Documentos/Mini-Radar-ENEM-com-Python-e-Docker-Aula-3-Modelos-de-servico-IaaS-PaaS-SaaS$ source /home/joao/Documentos/Mini-Radar-ENEM-com-P
ython-e-Docker-Aula-3-Modelos-de-servico-IaaS-PaaS-SaaS/.venv/bin/activate
(.venv) joao@JP-G5-5590:~/Documentos/Mini-Radar-ENEM-com-Python-e-Docker-Aula-3-Modelos-de-servico-IaaS-PaaS-SaaS$ curl -X POST http://localhost:5000/exibir_n
ota \
-H "Content-Type: application/json" \
-d '{"notas": [720.5, 680.0, 810.2, 640.8, 780.0]}'
{"mensagem": "Sua nota de corte \u00e9: 726.3"}
(.venv) joao@JP-G5-5590:~/Documentos/Mini-Radar-ENEM-com-Python-e-Docker-Aula-3-Modelos-de-servico-IaaS-PaaS-SaaS$
```

Figura 4 — Requisição de teste ao endpoint `/exibir_nota`, confirmando que o site principal (PaaS) consulta o microserviço de cálculo (FaaS) e retorna o resultado ao usuário.

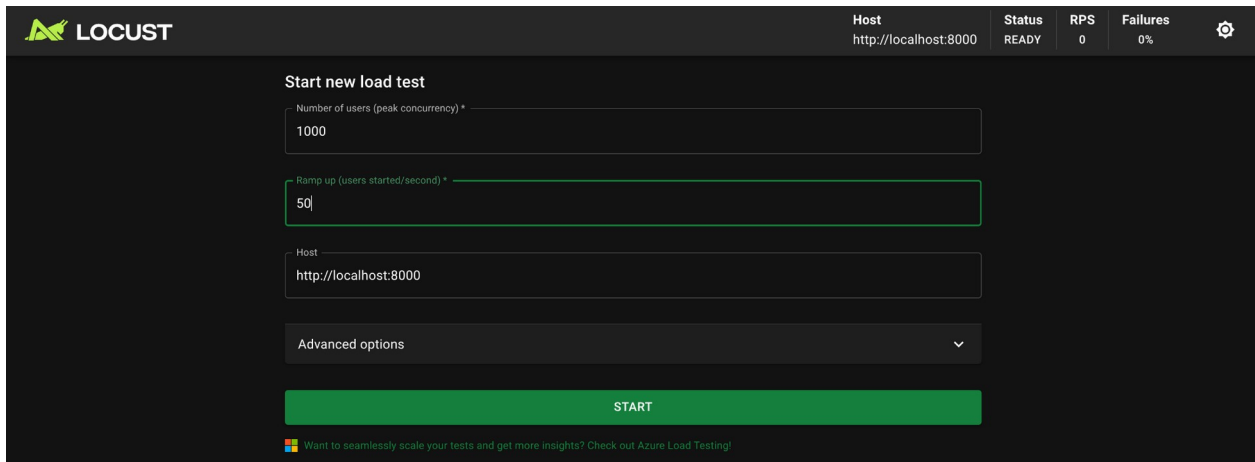
5. Fase 4 — Teste de Carga com Locust

Para avaliar a arquitetura sob pressão, foi utilizada a ferramenta **Locust**, que simula milhares de usuários acessando o sistema ao mesmo tempo:

```
locust -f locustfile.py --host http://localhost:8000
```

O teste foi configurado pela interface web (`http://localhost:8089`) com os seguintes parâmetros: **1000 usuários** (peak concurrency), **ramp up de 50 usuários por segundo** e host

http://localhost:8000.



LOCUST

Host: http://localhost:8000 | Status: READY | RPS: 0 | Failures: 0%

Start new load test

Number of users (peak concurrency) *
1000

Ramp up (users started/second) *
50

Host
http://localhost:8000

Advanced options ▾

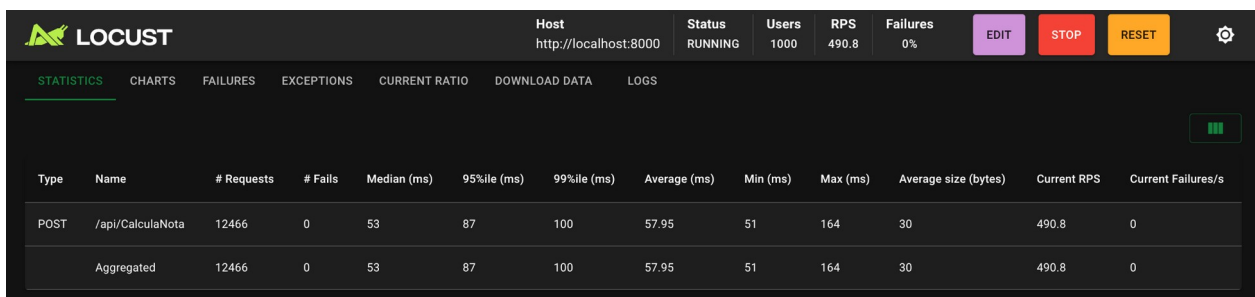
START

Want to seamlessly scale your tests and get more insights? Check out Azure Load Testing!

Figura 5 — Configuração do teste de carga na interface visual do Locust, simulando 1000 alunos acessando a calculadora de nota de corte simultaneamente.

Durante a execução, o painel **Statistics** exibiu os resultados agregados do endpoint POST /api/CalculaNota:

Métrica	Valor obtido
Requisições totais	12.466
Falhas	0 (0%)
Tempo de resposta (mediana)	53 ms
95º percentil	87 ms
99º percentil	100 ms
Tempo médio de resposta	57,95 ms
Mínimo / Máximo	51 ms / 164 ms
RPS (requisições por segundo)	490,8



LOCUST

Host: http://localhost:8000 | Status: RUNNING | Users: 1000 | RPS: 490.8 | Failures: 0% | EDIT | STOP | RESET | ⚙️

STATISTICS | CHARTS | FAILURES | EXCEPTIONS | CURRENT RATIO | DOWNLOAD DATA | LOGS

⏸️

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/api/CalculaNota	12466	0	53	87	100	57.95	51	164	30	490.8	0
Aggregated		12466	0	53	87	100	57.95	51	164	30	490.8	0

Figura 6 — Painel de estatísticas do Locust com o teste em execução (1000 usuários simultâneos, RPS de 490,8 e 0% de falhas), demonstrando que a calculadora escalada com 4 workers suportou a carga sem derrubar conexões.

6. Análise dos Resultados

- **RPS (Requisições por segundo):** a arquitetura sustentou uma média de **490,8 requisições por segundo** com 1000 usuários simultâneos, evidenciando o ganho de paralelismo obtido ao escalar a calculadora de 1 para 4 workers. Com apenas 1 worker, esse mesmo volume de usuários tenderia a gerar fila de espera e aumento expressivo da latência.
- **Response Times (Tempo de resposta):** a mediana de 53 ms e o 99º percentil de 100 ms mostram que, mesmo sob carga elevada, a grande maioria das requisições foi respondida rapidamente — resultado do processamento simulado no serviço somado ao paralelismo dos 4 workers.
- **Failed requests (Requisições falhas): 0% de falhas** em 12.466 requisições. Isso indica que, para o volume de carga testado, a arquitetura desacoplada (PaaS + FaaS) manteve-se estável, sem derrubar conexões.

7. Conclusão

O teste demonstrou, na prática, os benefícios da modernização da arquitetura:

- **Desacoplamento (PaaS + FaaS):** ao separar o site principal (Flask) da calculadora de nota de corte (FastAPI), cada parte passa a escalar e a ser monitorada de forma independente.
- **Escalabilidade horizontal:** o aumento de 1 para 4 workers no microserviço multiplicou a capacidade de atendimento, simulando o comportamento automático de escala de um provedor de nuvem real, como AWS Lambda ou Azure Functions.
- **Resiliência ativa:** o uso do parâmetro `timeout=2.0` e o tratamento das exceções `Timeout` e `ConnectionError` no `app.py` garantem que, mesmo se a calculadora ficar lenta ou indisponível, o site principal continua respondendo ao usuário com uma mensagem amigável, em vez de travar em cascata.

Esses resultados reforçam o conceito central da atividade: em um ambiente de nuvem moderno, a divisão de responsabilidades entre serviços — PaaS para a aplicação principal e FaaS para funções específicas — proporciona maior escalabilidade, resiliência e facilidade de manutenção quando comparada a uma arquitetura monolítica tradicional.